PayTrust: A Secure Peer-to-Peer Payment Platform with Conditional Escrow and Double-Entry Ledger Accounting

1. Abstract

PayTrust is a full-stack fintech web application. Users hold digital wallets, send money to each other instantly, and create **escrow-protected deals** in which the platform holds the buyer's money until the seller delivers. Every movement of money is recorded with **double-entry ledger accounting**, so the books always balance and any wallet can be re-derived from history.

MySQL stores everything that needs strict consistency: users, wallets, transactions, ledger entries and escrows. **MongoDB** stores flexible, document-shaped data: dispute evidence, KYC documents and activity logs. The front end and API are built with **Next.js**.

The project focuses on real backend problems that a normal CRUD app does not have: ACID transactions, concurrency control, idempotent APIs, a state machine enforced in the database, and the right use of two database types.

2. Domain

FinTech: digital payments and escrow infrastructure, modelled on systems such as PayPal, Venmo and Upwork's payment protection.

3. Problem Statement

In any exchange of money for a deliverable, one side must trust the other first. Direct peer-to-peer payment gives no protection if the other party defaults. Most student projects also store a balance as one editable number, which breaks under concurrent requests and cannot be audited.

PayTrust solves both problems. **Escrow** removes the trust problem, and a **double-entry ledger with row locking and idempotency keys** removes the correctness problem.

4. Objectives

1. Wallet-based peer-to-peer transfers with exact (DECIMAL) money arithmetic.
2. An escrow lifecycle (create, fund, complete, release, refund, dispute) enforced by stored procedures and database constraints.
3. Financial consistency through double-entry ledger accounting: every transaction has exactly one debit and one credit.
4. No double spending or lost updates under concurrent requests (row locks in a fixed order, idempotency keys).

5. A tamper-evident record: an append-only ledger and an audit trail of every escrow status change.
6. Dispute handling with admin arbitration, with evidence stored in MongoDB.
7. Role-based access (user and admin) and security best practice (hashed passwords, parameterised queries, no hard-coded secrets).

5. Scope

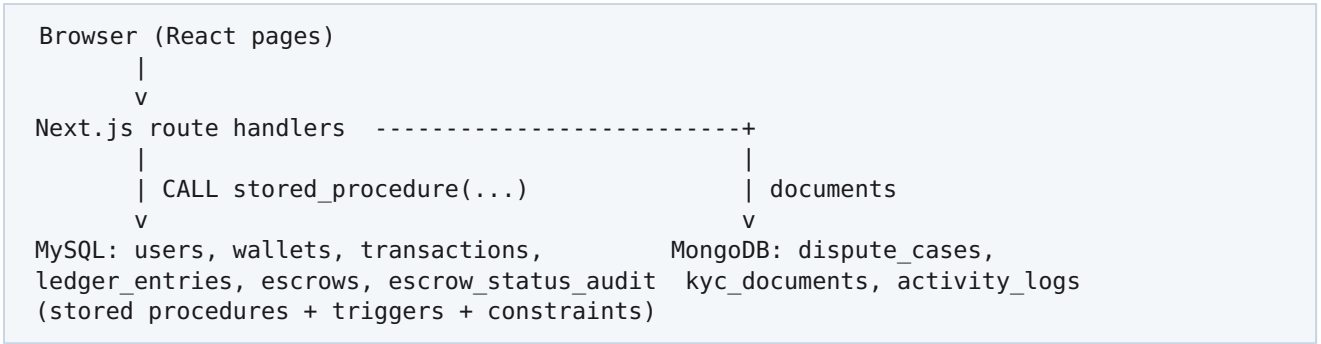
In scope: authentication, wallets, simulated top-ups, transfers, escrow, disputes, admin pages, transaction history, ledger reconciliation.

Out of scope: real payment gateways, real KYC checks and real banking rails. Funds are simulated through an "add funds" feature.

6. Technology Stack

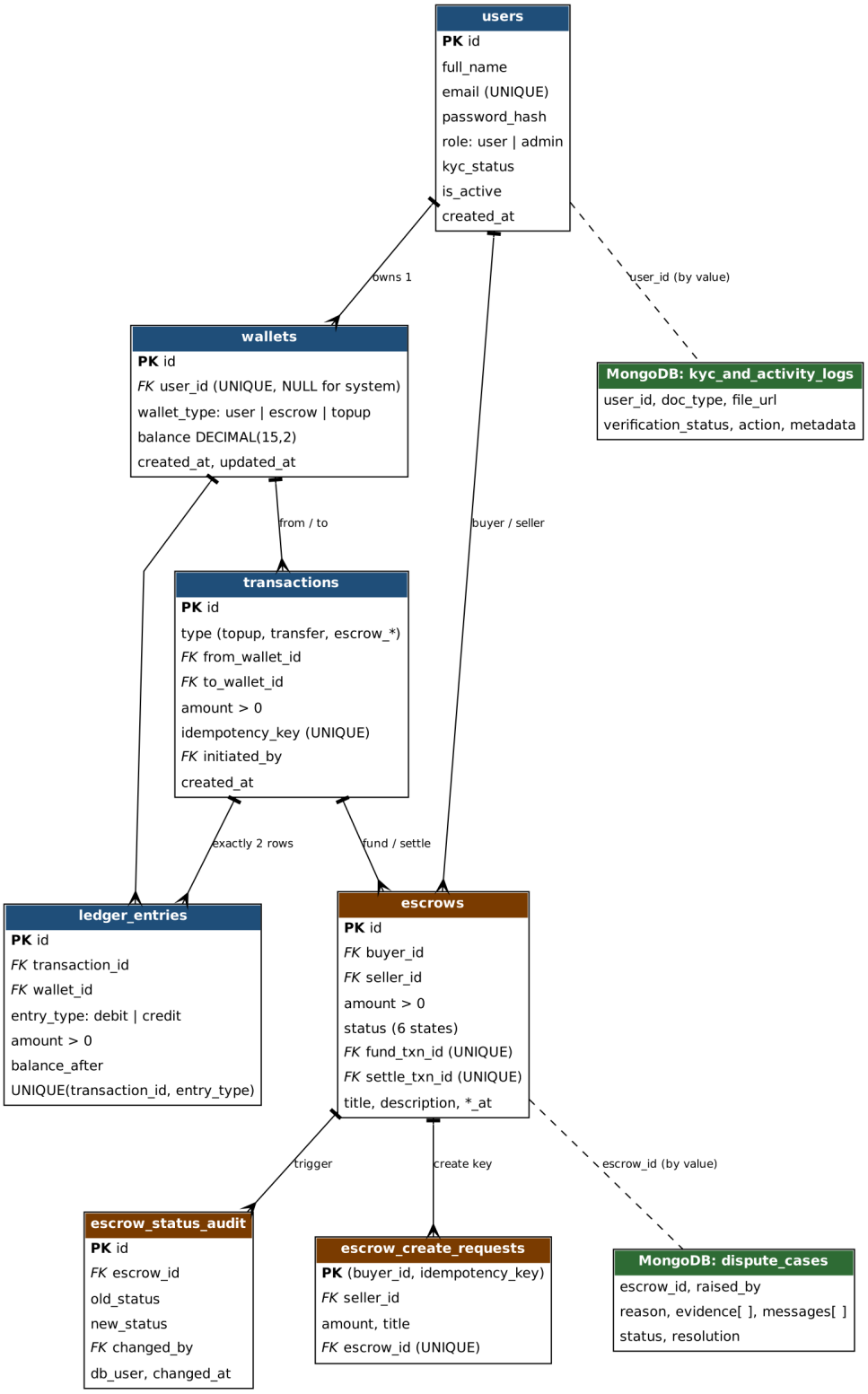
Layer	Technology
Front end and API	Next.js (App Router, route handlers), React
Relational database	MySQL 8, accessed with `mysql2` (parameterised queries, shared connection pool)
Non-relational database	MongoDB (disputes, KYC documents, activity logs)
Auth	Password hashing with bcrypt; role-based access (`user`, `admin`)
Configuration	Environment variables (`.env.local`); no credentials in code
Testing	Concurrency stress test (Node.js script), SQL checks, `curl` API tests
Version control	Git and GitHub, one branch per member, individual commits

7. Architecture



Key rule: the API never changes a balance itself. All money moves through MySQL stored procedures, so the rules live next to the data and cannot be bypassed by a bug in application code.

8. Database Design



Entity-relationship diagram. Blue = ledger tables, brown = escrow tables, green = MongoDB collections.

8.1 MySQL tables

All ids are `BIGINT UNSIGNED`. All money columns are `DECIMAL(15,2)`.

users

Column	Type / rule
id	PK
full_name	VARCHAR(100), must not be blank
email	VARCHAR(150), UNIQUE, CHECK on shape
password_hash	VARCHAR(255)
role	ENUM('user','admin')
kyc_status	ENUM('pending','verified','rejected')
is_active	BOOLEAN (admin can freeze an account)
created_at	DATETIME(3)

wallets

Column	Type / rule
id	PK
user_id	FK to users, UNIQUE (one wallet per user), NULL for system wallets
wallet_type	ENUM('user','escrow','topup')
balance	DECIMAL(15,2), `CHECK (balance >= 0 OR wallet_type = 'topup')`
created_at, updated_at	DATETIME(3)

Two system wallets exist: **escrow** (holds money while an escrow is funded) and **topup** (the source of simulated deposits; it goes negative by design, so the sum of all wallets is always exactly 0).

transactions (one row per money movement)

Column	Type / rule
id	PK
type	ENUM('topup','transfer','escrow_fund','escrow_release','escrow_refund')
from_wallet_id, to_wallet_id	FKs to wallets, must differ
amount	DECIMAL(15,2), `CHECK (amount > 0)`
idempotency_key	VARCHAR(64), **UNIQUE**
initiated_by	FK to users
created_at	DATETIME(3)

ledger_entries (the double-entry book, append-only)

Column	Type / rule
id	PK
transaction_id	FK to transactions
wallet_id	FK to wallets
entry_type	ENUM('debit','credit')
amount	DECIMAL(15,2), `CHECK (amount > 0)`
balance_after	DECIMAL(15,2), wallet balance right after this entry
(constraint)	`UNIQUE (transaction_id, entry_type)`: at most one debit and one credit per transaction

escrows

Column	Type / rule
id	PK
buyer_id, seller_id	FKs to users, `CHECK (buyer_id <> seller_id)`
amount	DECIMAL(15,2), `CHECK (amount > 0)`
title, description	text
status	ENUM('created','funded','completed','released','refunded','disputed')
fund_txn_id, settle_txn_id	FKs to transactions, UNIQUE
(constraints)	status other than `created` requires `fund_txn_id`; `released` or `refunded` requires `settle_txn_id`
funded_at, completed_at, settled_at, updated_at	DATETIME(3)

escrow_create_requests (makes "create escrow" safe to retry)

Column	Type / rule
buyer_id, idempotency_key	composite PK; buyer_id FK to users
seller_id	FK to users
amount, title	the details of the request, to detect a key reused for a different escrow
escrow_id	FK to escrows, UNIQUE
created_at	DATETIME(3)

escrow_status_audit (filled by a trigger): escrow_id, old_status, new_status, changed_by, db_user, changed_at.

View `wallet_balance_summary`: for every wallet, the cached balance, the balance recomputed from `ledger_entries`, and their difference. A healthy system shows a difference of 0 everywhere.

8.2 MongoDB collections

- **dispute_cases**: escrow_id, raised_by, against, reason, description, status, evidence array (files), messages array, resolution (decision, resolved_by, resolved_at, note). Evidence differs from case to case, so a document fits better than fixed columns.
- **kyc_documents**: user_id, doc_type, masked document number, file_url, status, rejection_reason.
- **activity_logs**: user_id, action, entity_id, ip, metadata, created_at. High-volume, schema-flexible log data.

The link between the two databases is by value (`escrow_id`, `user_id`), as is normal for polyglot persistence.

9. Money Flow and Stored Procedures

Every procedure runs in one database transaction, locks the wallet rows it needs, and either fully succeeds or leaves no trace.

Procedure	What it does	Who may call it
`add_funds(user_id, amount, key)`	topup wallet to user wallet	the user (or admin)
`process_transfer(from, to, amount, key)`	user wallet to user wallet	owner of the source wallet
`create_escrow(seller_email, amount, title, description, key)`	opens a new escrow in `created`; no money moves; idempotent per buyer	any signed-in user except admin
`fund_escrow(id, key)`	buyer wallet to escrow wallet; `created` to `funded`	buyer
`complete_escrow(id)`	seller says work is done; `funded` to `completed` (no money moves)	seller
`release_escrow(id, key)`	escrow wallet to seller wallet; `completed` to `released`	buyer (disputed: admin)
`refund_escrow(id, key)`	escrow wallet to buyer wallet; to `refunded`	seller (disputed: admin)
`dispute_escrow(id)`	freezes the escrow; `funded`/ `completed` to `disputed`	buyer or seller
`resolve_dispute(id, decision, key)`	admin’s decision on a disputed escrow: release to seller or refund to buyer	admin

Escrow state machine

```
created --fund--> funded --complete--> completed --release--> released
                    |                      |
                    +---- refund ---------+--> refunded
                    +---- dispute --------+--> disputed --(admin)--> released | refunded
```

Each money movement inserts one `transactions` row, updates two wallet balances and inserts exactly two `ledger_entries` rows (one debit, one credit) with the same amount.

10. Concurrency, Idempotency and Integrity

1. **Row locking in a fixed order.** Wallets are always locked lowest id first, and an escrow row is locked before its wallets. Two requests can never wait on each other in a circle, so deadlocks cannot happen.
2. **Idempotency keys.** Each money request carries a unique key. Repeating a request with the same key returns the original result (`replayed: true`) and moves nothing. Reusing a key for a different request is rejected (`IDEMPOTENCY_CONFLICT`). This protects against double clicks and network retries.
3. **Overdraft protection** in two layers: the procedure checks the balance after locking the row, and a `CHECK` constraint on the table is the last line of defence.
4. **Append-only ledger.** Triggers reject every `UPDATE` and `DELETE` on `ledger_entries` and `transactions`. A mistake is corrected by a new, opposite transaction.
5. **Audit trail.** A trigger on `escrows` writes every status change, with the acting user and database user, into `escrow_status_audit`.
6. **Actor enforcement.** The acting user is passed to the database in a session variable. Procedures refuse to run without a valid, active user and check role and ownership themselves.
7. **Consistent errors.** Every failure carries a code the API turns into an HTTP status:

Code	HTTP	Meaning
INVALID_INPUT	400	bad amount, id or missing key
FORBIDDEN	403	not allowed or account frozen
NOT_FOUND	404	wallet or escrow missing
INVALID_TRANSITION	409	escrow is in the wrong state
IDEMPOTENCY_CONFLICT	409	key reused for another request
INSUFFICIENT_BALANCE	422	wallet cannot cover the amount

11. Advanced Database Features Used

Feature	Where
Stored procedures with transactions and error signalling	all money movement
Triggers	escrow status audit; append-only ledger guards
Views	`wallet_balance_summary` (reconciliation)
Constraints (CHECK, UNIQUE, FK)	non-negative balances, positive amounts, one debit and one credit per transaction, state-dependent escrow columns
Row-level locking and isolation control	`SELECT ... FOR UPDATE`, READ COMMITTED inside procedures
Indexes	statement lookup `(wallet_id, created_at, id)`, escrow status and party indexes
Window functions and CTEs	wallet statement with running balance; monthly top senders ranking (`db/queries_a.sql`)

12. API Endpoints (money and ledger)

Amounts are strings (`"500.00"`). Every money call requires an `Idempotency-Key` header.

Method and path	Purpose
POST `/api/wallet/add-funds`	simulated top-up
POST `/api/transfer`	send money to another user's wallet
POST `/api/escrow`	create an escrow (`seller_email, amount, title, description`)
POST `/api/escrow/:id/fund`	buyer funds an escrow
POST `/api/escrow/:id/complete`	seller marks work done
POST `/api/escrow/:id/release`	buyer releases money to seller
POST `/api/escrow/:id/refund`	refund the buyer
POST `/api/escrow/:id/dispute`	freeze the escrow
GET `/api/admin/reconciliation`	admin: wallet-by-wallet ledger check

Login and registration (JWT session cookie), dispute cases (MongoDB), KYC, history and the remaining admin pages are provided by the other team member's modules.

13. Testing and Results

Functional checks (SQL): transfers, insufficient balance, idempotent replay, idempotency key reuse with different data, wrong user, wrong escrow state, admin-only dispute resolution, and edits to the ledger being rejected. All behaved as designed.

Concurrency stress test (`scripts/stress_test.mjs`, run against a separate test database):

Check	Result
200 simultaneous transfers in opposite directions between two wallets	all 200 succeeded, no deadlock, balances exact
50 simultaneous requests with the same idempotency key	exactly 1 transaction created
30 simultaneous spends from a wallet that could afford 10	exactly 10 succeeded, 20 rejected with INSUFFICIENT_BALANCE
20 simultaneous releases of one escrow	exactly 1 succeeded
Sum of all wallet balances	0.00
Cached balance vs ledger-derived balance, every wallet	difference 0
Every transaction has exactly one debit and one credit of equal amount	yes

Reconciliation page: the admin page shows each wallet's cached balance, ledger-derived balance and difference, with a clear "Healthy" or "Mismatch" badge.

14. Sample Data

The project ships with real demo data, loaded by the seed scripts (or in one step by `paytrust_full.sql`). Every rupee in it was created by the stored procedures, so the ledger reconciles.

Table	Rows	Notes
users	26	1 admin, 25 users; KYC: 18 verified, 5 pending, 3 rejected; 1 frozen account
wallets	27	one per user, plus the escrow and top-up system wallets
transactions	119	31 top-ups, 47 transfers, 26 escrow funds, 11 releases, 4 refunds
ledger_entries	238	exactly 2 per transaction
escrows	34	8 created, 6 funded, 4 completed, 11 released, 4 refunded, 1 disputed
escrow_status_audit	96	one row per status change, with the acting user
escrow_create_requests	28	one per escrow created through the API

Demo login for every user: `<first name>@paytrust.test` with password `Demo@1234` (for example `aarav@paytrust.test`, admin: `admin@paytrust.test`). MongoDB holds the demo dispute case, KYC submissions and activity logs.

15. Division of Work

Member A	Member B
Ledger tables and reconciliation view	Escrow tables and status audit trigger
Money stored procedures, create-escrow procedure, append-only triggers	Seed data, MongoDB setup and analytics
Add funds, transfer, create-escrow and escrow action APIs, idempotency, escrow action buttons	Authentication, roles, dispute API and resolution
Reconciliation page, concurrency stress test, bulk demo data	KYC, history, search, remaining pages and admin analytics

16. Future Scope

- Real payment gateway in test mode (Razorpay or Stripe)
- Multi-currency wallets
- Automatic release when a buyer does not respond in time
- Fraud detection on transaction patterns
- Scheduled and recurring payments

17. Why This Project Stands Out

- **Financial correctness:** a real double-entry ledger that can prove itself, not a balance column.
- **Concurrency control:** a published stress test that anyone can run and watch pass.
- **Rules in the database:** the escrow state machine, ownership checks and append-only history cannot be bypassed by application bugs.
- **Polyglot persistence used for a reason:** MongoDB holds the data that is a poor fit for fixed tables.